

Advanced Graph Workflows: Custom and Complex State Definitions

1. Introduction

In LangGraph, **State** represents the data shared between nodes in a graph.

For a simple order-processing workflow, the state might contain:

```
class OrderState(TypedDict):  
    order_id: str  
    status: str
```

As the workflow becomes more complex, multiple nodes may need to update the same state key. LangGraph provides **reducers** to control how these overlapping updates are combined.

2. State as Shared Data

Consider this workflow:

```
START  
|  
v  
Validate Order  
|  
+----> Check Inventory  
|  
+----> Check Payment  
|  
v  
Process Order
```

The nodes operate on a shared **OrderState**.

For example, the validation node might update:

```
return {"status": "VALID"}
```

The inventory node might update:

```
return {"inventory": {"available": True}}
```

A node only needs to return the fields it wants to update. It does not have to return the complete state.

3. Overlapping State Keys

Suppose the state contains:

```
messages: list[str]
```

Both the inventory and payment nodes want to add information to `messages`.

```
Inventory Node
  |
  +----> messages

Payment Node
  |
  +----> messages
```

For example:

```
# Inventory node
return {
    "messages": ["Inventory available"]
}
```

and:

```
# Payment node
return {
    "messages": ["Payment successful"]
}
```

Now there are two updates to the same key.

The question is:

How should LangGraph combine these updates?

This is where a **reducer** is used.

4. Reducers

A reducer defines how a new value should be combined with the existing value.

For example:

```
from typing import Annotated
import operator

messages: Annotated[
    list[str],
    operator.add
]
```

Conceptually:

```
Existing messages
      +
New messages
      |
      v
    Reducer
      |
      v
Combined messages
```

So:

```
["Order validated"]
+
["Inventory available"]
```

can become:

```
[
    "Order validated",
    "Inventory available"
]
```

The reducer therefore controls the update behavior of a particular state key.

5. Normal Keys vs. Reducer Keys

Consider a normal state field:

```
status: str
```

If the current value is:

NEW

and a node returns:

VALID

the new value becomes:

VALID

The previous value is replaced.

A reducer-based field behaves differently:

```
messages: Annotated[
    list[str],
    operator.add
]
```

Updates are combined according to the reducer.

Therefore:

Normal key:

old value → new value

Reducer key:

old value + new value → reducer(old, new)

This distinction is fundamental when designing LangGraph state.

6. Custom Reducers

A reducer does not have to use `operator.add`.

We can define our own merging logic.

For example:

```
def combine_messages(old, new):
    result = old.copy()

    for message in new:
        if message not in result:
            result.append(message)

    return result
```

The state can then use:

```
messages: Annotated[
    list[str],
    combine_messages
]
```

Now duplicate messages are removed when updates are combined.

This is useful when the application needs domain-specific merging behavior.

7. Complex State

A real workflow can have several different state fields:

```
class OrderState(TypedDict):
    order_id: str
    customer: dict
    status: str
    inventory: dict
    payment: dict

    messages: Annotated[
        list[str],
        operator.add
    ]
```

Different nodes can update different parts of this state.

For example:

```
OrderState
|
+-- status
```

```
|
+-- inventory
|
+-- payment
|
+-- messages
```

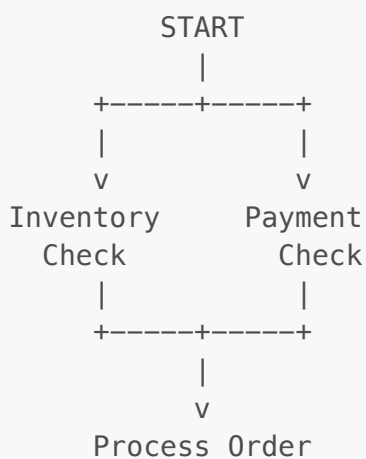
The inventory node can update **inventory** and **messages**, while the payment node can update **payment** and **messages**.

The reducer controls how their overlapping **messages** updates are combined.

8. Why Reducers Matter in Graph Workflows

Reducers become particularly useful when graph nodes execute independently or in parallel.

Consider:



Both nodes can produce messages.

Without a suitable reducer, multiple updates to the same key may not produce the desired combined result.

With a reducer:



The reducer gives the graph a well-defined rule for merging the updates.

9. Designing State Carefully

When defining complex state, consider each field independently:

Is it a single value?

For example:

```
status: str
```

Usually, the latest update should replace the previous value.

Can multiple nodes contribute?

For example:

```
messages: list[str]
```

A reducer may be appropriate.

Does the application require special merging?

For example:

```

Combine lists
Remove duplicates
Merge dictionaries
Keep the highest priority value
Preserve the first value
  
```

A custom reducer may be appropriate.

10. Key Takeaway

LangGraph state is more than a container for variables.

A state definition determines both:

1. **What data flows through the graph**
2. **How updates to that data are handled**

The basic model is:

```
State
|
+----- Normal key
|           |
|           +--> New value replaces old value
|
+----- Reducer key
|           |
|           +--> Updates are merged
```

The central idea is:

Reducers define how LangGraph should combine multiple updates to the same state key.

This becomes especially important when building advanced workflows with **shared state, overlapping updates, and parallel nodes**.